

AI Operations (AIOps)

Module 2: Distributed Computing for Data & Models

Course Content — 3 Lectures (9 Contact Hours) | Third-Year B.Sc./B.Tech. Data Science

Module Overview

Module 1 established that models must be reproducible and auditable. Module 2 tackles the next wall every AI team hits: data and compute outgrow a single machine. Students learn to process data at scale with Apache Spark, run distributed Python compute and training with Ray, and close the loop with two production-critical disciplines that keep distributed pipelines trustworthy: Feature Stores (Feast) for consistent features between training and serving, and automated Data Quality Validation (Great Expectations) for catching drift and bad data before it reaches a model.

Lecture	Title	Duration	Core Focus
L1	Distributed Data Processing with Apache Spark	~3.0 hrs	RDDs vs. DataFrames; transformations/actions; partitioning & shuffles; large-scale ETL
L2	Distributed Compute & Training with Ray	~3.0 hrs	Ray Core (tasks/actors); Ray Train; data vs. model parallelism; fault tolerance
L3	Feature Stores (Feast) & Data Quality (Great Expectations)	~3.0 hrs	Offline/online feature stores; training-serving skew; expectation suites; drift detection

Module Learning Outcomes

- Explain why a single machine cannot process modern training datasets, and how Spark's cluster model (driver/executors) solves this (CO3, CO4).
- Distinguish RDDs, DataFrames, and Spark SQL, and choose the right abstraction for a given ETL task (CO4).
- Reason about lazy evaluation, transformations vs. actions, and partitioning/shuffle costs when designing a Spark pipeline (CO4).
- Write distributed Python programs using Ray tasks and actors, and scale a training job with Ray Train (CO3, CO4).
- Differentiate data parallelism from model parallelism and select the appropriate strategy for a given model/cluster size (CO4).
- Design a feature store layer that eliminates training-serving skew, using Feast's offline and online stores (CO4, CO7).
- Build automated data validation suites with Great Expectations to detect data and concept drift before they reach production (CO6).

Lecture 1 — Distributed Data Processing with Apache Spark (≈3 hours)

1.1 Why Distributed Data Processing?

A modern recommendation engine or fraud model routinely trains on hundreds of millions of rows spanning terabytes. A single machine, however large, eventually runs out of RAM, disk I/O bandwidth, or simply takes too long. Apache Spark solves this by splitting data across a cluster of machines and running the same computation on each partition in parallel, then combining results.

Discussion prompt for class

Ask students: “You have a 500 GB clickstream log and a laptop with 16 GB RAM. Pandas' `read_csv` will not even load this file. What are your options?” Lead them toward: (a) don't load it all at once (chunking), (b) use a machine with more RAM (vertical scaling — has a ceiling), (c) split the file across many machines and process in parallel (horizontal scaling — what Spark does).

1.2 Spark Architecture

- **Driver** — the process running your main program; builds the execution plan (the DAG) and coordinates the cluster.
- **Cluster Manager** — allocates resources across the cluster (standalone, YARN, Kubernetes, or Mesos).
- **Executors** — worker processes on cluster nodes that actually run tasks and store data partitions in memory/disk.
- **Tasks** — the smallest unit of work; one task processes one partition of data.

Understanding this split matters operationally: a job that is slow because the Driver is overloaded (e.g., collecting too much data to it) has a completely different fix than a job that is slow because of network shuffles between Executors.

1.3 RDDs vs. DataFrames vs. Spark SQL

Abstraction	What It Is	When to Use
RDD (Resilient Distributed Dataset)	Low-level distributed collection of Python/Java/Scala objects, no schema	Rarely needed directly today; useful for unstructured data or full control over partitioning
DataFrame	Distributed table with a schema (columns + types), like a distributed pandas DataFrame	The default choice for almost all ETL and feature engineering — optimized by Spark's Catalyst engine
Spark SQL	Write standard SQL directly against DataFrames registered as temporary views	Ideal for analysts/teams already fluent in SQL; identical execution engine as DataFrames

Because DataFrames and Spark SQL share the same optimized execution engine (Catalyst) and the same physical plan optimizer (Tungsten), there is no performance penalty for preferring whichever syntax is more readable for a given task — unlike raw RDDs, which bypass these optimizations entirely.

1.4 Lazy Evaluation: Transformations vs. Actions

Spark code does not execute the moment you write it. Instead, DataFrame operations fall into two categories:

- **Transformations (e.g., `.filter()`, `.select()`, `.groupBy()`, `.join()`) — lazily build up a logical execution plan (a DAG) without touching data.**
- **Actions (e.g., `.count()`, `.collect()`, `.write()`, `.show()`) — trigger actual execution of the entire accumulated DAG.**

This matters practically: a chain of ten `.filter()` and `.withColumn()` calls costs nothing until an action runs, and Spark's optimizer can reorder and fuse those transformations for efficiency before executing them — something impossible in eager, row-by-row pandas code.

1.5 Partitioning & Shuffles

- A DataFrame is split into partitions, each processed independently by one task on one executor core.
- Operations like `groupBy`, `join`, and `repartition` can require a shuffle — redistributing data across the network so that related rows land on the same executor.
- Shuffles are the single most expensive operation in Spark: they involve disk I/O, serialization, and network transfer across the whole cluster.
- Practical levers: choosing a sensible number of partitions (too few → underutilized cluster; too many → scheduling overhead), and using broadcast joins for a small table joined against a huge one (avoids shuffling the huge table at all).

1.6 Case Study: Scaling a Recommendation Engine's Feature Pipeline

Scenario: A university course-recommendation startup's feature pipeline (computing “clicks per course per week,” “average session length,” etc.) runs fine on a pandas script with 50,000 users. At 5 million users, the nightly job that used to take 20 minutes now takes 14 hours and frequently runs out of memory.

Guided in-class walkthrough

1. Identify the bottleneck: pandas loads the entire dataset into a single process's memory — there is no horizontal scaling path.
2. Migrate the pipeline to Spark DataFrames: the same `groupBy/aggregate` logic now runs across many executors in parallel.
3. Address shuffle cost: the `groupBy("course_id")` aggregation requires a shuffle — discuss partitioning strategy (partition by `course_id` upstream, when writing the raw event logs, to minimize shuffle volume later).
4. Discuss output format: writing results as partitioned Parquet files (columnar, compressed, splittable) instead of CSV, so downstream jobs and even individual columns can be read selectively.

1.7 Hands-on Lab: Large-Scale ETL with PySpark

Objective: process a large (synthetic, 10GB+ scale) clickstream dataset entirely with Spark DataFrames, compute aggregated features, and write the result as partitioned Parquet.

Step 1 — Start a local Spark session

```

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("aiops-etl-lab")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate()
)

```

Step 2 — Read raw data and inspect the execution plan

```

df = spark.read.parquet("data/clickstream/")
df.printSchema()
df.explain(True) # shows the DAG BEFORE any action runs

```

Step 3 — Transform: filter, aggregate, join (all lazy)

```

from pyspark.sql import functions as F

features = (
    df.filter(F.col("event_type") == "click")
    .groupBy("user_id", "course_id")
    .agg(
        F.count("*").alias("click_count"),
        F.avg("session_length_sec").alias("avg_session_length"),
    )
)

```

Step 4 — Trigger execution and write partitioned output

```

features.write.mode("overwrite").partitionBy("course_id").parquet("output/features/")
print("Rows written:", features.count()) # an ACTION

```

Step 5 — Compare a naive vs. broadcast join (pair exercise)

```

small_lookup_df = spark.read.parquet("data/course_catalog/") # small table

# Naive join -- may shuffle the large table
joined_naive = features.join(small_lookup_df, "course_id")

# Broadcast join -- sends the small table to every executor, avoids shuffling `features`
from pyspark.sql.functions import broadcast
joined_fast = features.join(broadcast(small_lookup_df), "course_id")

```

Each pair compares the physical plans (.explain()) and wall-clock time of both joins, and documents which shuffle stages disappear with the broadcast hint.

Lecture 2 — Distributed Compute & Training with Ray (≈3 hours)

2.1 Why Ray, When We Already Have Spark?

Spark excels at large, mostly-tabular ETL with a mature SQL-like API. But modern AI workloads increasingly need to distribute arbitrary Python code — custom training loops, reinforcement learning simulations, hyperparameter sweeps, and multi-GPU model training — which do not map cleanly onto Spark's DataFrame model. Ray fills this gap: a general-purpose distributed computing framework for scaling ordinary Python functions and classes with minimal code changes.

2.2 Ray Core: Tasks and Actors

- **Task** (`@ray.remote` on a function) — a stateless unit of work; calling it returns immediately with a future (an `ObjectRef`), and Ray schedules the actual execution on any available worker.
- **Actor** (`@ray.remote` on a class) — a stateful, long-lived worker process; useful when you need to hold state (a loaded model, a simulation environment) across many calls.
- **Object Store** — Ray's shared-memory store for large objects (e.g., a NumPy array), letting multiple tasks/actors access the same data without repeated serialization.

The key mental model

A normal Python function call blocks and returns a value. A Ray task call (`my_func.remote(...)`) returns immediately with a reference to a future result; you call `ray.get()` when you actually need the value. This is what allows many tasks to be launched concurrently across a cluster with ordinary-looking Python code.

2.3 Ray Train: Data Parallelism vs. Model Parallelism

Strategy	What Is Split	When to Use
Data Parallelism	The DATASET is split across workers; each worker holds a full copy of the model	The model fits in a single GPU's memory, but training is slow because the dataset is huge
Model Parallelism	The MODEL itself is split across workers (different layers or tensor shards per device)	The model itself is too large to fit in a single GPU's memory (e.g., large language models)

Ray Train provides a unified API for both: it manages worker processes, handles gradient synchronization for data-parallel training, and integrates with PyTorch's Distributed Data Parallel (DDP) and Fully Sharded Data Parallel (FSDP) for model parallelism, without students needing to hand-roll multiprocessing or NCCL setup themselves.

2.4 Fault Tolerance in Distributed Training

- A training job running across 32 GPUs for 3 days has a meaningfully higher chance that at least one node fails mid-run than a single-machine job.
- Ray Train supports automatic checkpointing and worker recovery: if a worker dies, Ray can restart it and resume from the last checkpoint rather than restarting the entire job.
- This connects directly back to Module 1: the checkpoint itself should be logged with MLflow so a resumed run remains fully traceable.

2.5 Case Study: Training a Large Language Model Across a Multi-Node GPU Cluster

Scenario: a research team needs to fine-tune a multi-billion-parameter language model that does not fit on a single GPU's memory, using a cluster of 8 nodes with 4 GPUs each (32 GPUs total).

Guided in-class walkthrough

5. Confirm the constraint: the model's parameters + optimizer states + activations exceed a single GPU's VRAM — pure data parallelism (copying the whole model to every GPU) is not possible.
6. Introduce model parallelism: shard the model's layers/tensors across GPUs (tensor or pipeline parallelism).
7. Combine strategies: use model parallelism WITHIN a node (fast NVLink interconnect) and data parallelism ACROSS nodes (slower network) — a hybrid approach used by virtually all large-scale LLM training today.
8. Discuss fault tolerance: with 32 GPUs running for days, checkpointing frequency becomes a cost/risk tradeoff — too frequent wastes I/O bandwidth, too infrequent risks losing significant compute on failure.

2.6 Case Study: Cost-Optimization of GPU Clusters Using Spot Instances

Spot/preemptible instances cost a fraction of on-demand GPU pricing but can be reclaimed by the cloud provider with little warning. Ray's fault-tolerant task/actor model and Ray Train's checkpoint-and-resume capability make this economically viable: a job can lose a spot-instance worker mid-training and simply resume on a new one, rather than losing the entire run.

2.7 Hands-on Lab: Ray Tasks, Actors & Distributed Training

Step 1 — Initialize Ray and launch parallel tasks

```
import ray
ray.init()

@ray.remote
def process_chunk(chunk_id, size=1_000_000):
    import numpy as np
    data = np.random.rand(size)
    return chunk_id, float(data.mean())

futures = [process_chunk.remote(i) for i in range(8)]
results = ray.get(futures) # blocks until all 8 tasks finish
print(results)
```

Step 2 — Use an Actor to hold state across calls

```
@ray.remote
class RunningStats:
    def __init__(self):
        self.total = 0
        self.count = 0

    def add(self, value):
```

```

        self.total += value
        self.count += 1
        return self.total / self.count

stats = RunningStats.remote()
for v in [10, 20, 30]:
    avg = ray.get(stats.add.remote(v))
print("Running average:", avg)

```

Step 3 — Train a model across multiple workers with Ray Train

```

from ray.train.torch import TorchTrainer
from ray.train import ScalingConfig

def train_loop_per_worker(config):
    import torch, ray.train
    model = build_model()
    optimizer = torch.optim.Adam(model.parameters(), lr=config["lr"])
    for epoch in range(config["epochs"]):
        loss = train_one_epoch(model, optimizer)
        ray.train.report({"loss": loss, "epoch": epoch})

trainer = TorchTrainer(
    train_loop_per_worker,
    train_loop_config={"lr": 1e-3, "epochs": 5},
    scaling_config=ScalingConfig(num_workers=4, use_gpu=False),
)
result = trainer.fit()

```

Lab target: every student launches at least 8 concurrent Ray tasks, one stateful actor, and one (CPU-only, small) Ray Train job across 2–4 simulated workers, observing the per-worker loss logs.

Lecture 3 — Feature Stores (Feast) & Data Quality Validation (Great Expectations) (≈3 hours)

3.1 The Training-Serving Skew Problem

A model is trained offline using a batch feature pipeline (e.g., “average purchases in the last 30 days,” computed nightly in Spark). At inference time, a real-time serving system must compute the SAME feature, but now instantly, from a live event stream. If these two computations drift apart — different code, different rounding, different time windows — the model silently sees different feature distributions in production than it was trained on. This is training-serving skew, and it is one of the most common causes of “the model was 95% accurate offline but performs badly in production.”

Discussion prompt for class

Ask students: “Your offline pipeline computes `avg_purchase_30d` using calendar days. Your real-time service computes it using a rolling 30x24x60 minute window. Are these the same number?” Usually not exactly — a concrete, memorable illustration of skew.

3.2 Feature Stores: The Feast Architecture

- **Entity** — the object a feature describes (e.g., `user_id`, `course_id`).
- **Feature View** — a named group of features tied to an entity and a data source, with a schema.
- **Offline Store** — a historical, point-in-time-correct store (often the same warehouse/data lake used for Spark ETL) used to build TRAINING datasets.
- **Online Store** — a low-latency key-value store (e.g., Redis, DynamoDB) used to serve the LATEST feature values at inference time.
- **Feature Service** — a named, versioned bundle of feature views that a specific model consumes — the contract between feature engineering and model serving.

The critical guarantee Feast provides: the SAME feature definition (the same transformation code) populates both the offline store (for training) and the online store (for serving), directly eliminating the class of skew described above.

3.3 Point-in-Time Correctness

When building a training dataset, it is essential that each row's features reflect only information available BEFORE that row's label timestamp — otherwise the model trains on “future” information it will never have at real inference time (a subtle form of label leakage). Feast's offline store performs point-in-time joins automatically: for each training example, it looks up the feature values as they existed at that example's timestamp, not the latest values.

3.4 Case Study: A Real-Time Feature Pipeline for a Recommendation Engine

Scenario: the course-recommendation model (Lecture 1's case study) needs a feature — “clicks in the last 10 minutes” — that is meaningless if computed nightly in Spark; it must reflect near-live behavior.

Guided walkthrough

9. Define the feature view once: `click_count_10m`, sourced from the same event stream Lecture 1 processed in batch for historical features.

10. Stream the raw events into the Online Store continuously (e.g., via a streaming job) so the value is always fresh at request time.
11. At training time, Feast reconstructs historical values of this same feature from the Offline Store, with point-in-time correctness, so training and serving are guaranteed consistent.
12. At serving time, the recommendation service performs a single low-latency lookup against the Online Store — no recomputation, no separate code path.

3.5 Data Quality Validation with Great Expectations

- **Expectation** — a single, testable assertion about data, e.g., `expect_column_values_to_not_be_null` or `expect_column_mean_to_be_between`.
- **Expectation Suite** — a named collection of expectations for a specific dataset/table.
- **Checkpoint** — a configured, repeatable way to run a suite against a batch of data (e.g., every time a new Spark ETL run completes).
- **Data Docs** — auto-generated, human-readable HTML reports showing which expectations passed/failed, with visualizations — a shareable audit artifact.

3.6 Case Study: Detecting Data Drift in a Credit Scoring System

Scenario: a credit-scoring model's approval rate for a demographic segment shifts unexpectedly over three months. Nobody changed the model or its code.

Guided walkthrough

13. Without automated validation, this drift is discovered only when someone notices the business impact — potentially months later, with regulatory exposure (recall Module 1's healthcare audit case study — the same governance stakes apply here).
14. With a Great Expectations checkpoint running on every batch of new applications, an expectation like `expect_column_mean_to_be_between("applicant_income", min, max)` or a distribution-comparison expectation would flag the shift within one pipeline run.
15. Discuss the difference between a hard failure (block the pipeline, e.g., nulls in a required field) and a soft warning (log and alert but continue, e.g., a mean shifting by an unusual amount) — both are legitimate checkpoint actions, chosen per expectation based on business risk.
16. Connect back to Lecture 1's Module 1 concept of Concept Drift and Data Drift: Great Expectations operationalizes the DETECTION half of the problem; retraining pipelines (Module 4/6) operationalize the RESPONSE half.

3.7 Hands-on Lab: Building a Feature Store and a Validation Suite

Part A — Feast: define and materialize a feature view

```
# feature_repo/features.py
from feast import Entity, FeatureView, Field, FileSource
from feast.types import Float32, Int64
from datetime import timedelta

user = Entity(name="user_id", join_keys=["user_id"])

clicks_source = FileSource(
```

```

    path="data/user_click_features.parquet",
    timestamp_field="event_timestamp",
)

user_click_stats = FeatureView(
    name="user_click_stats",
    entities=[user],
    ttl=timedelta(days=30),
    schema=[
        Field(name="click_count_7d", dtype=Int64),
        Field(name="avg_session_length", dtype=Float32),
    ],
    source=clicks_source,
)

$ feast apply          # registers entities/feature views
$ feast materialize-incremental $(date +%Y-%m-%d) # populates the online store

```

Part A — Retrieve features for training vs. serving

```

from feast import FeatureStore
store = FeatureStore(repo_path="feature_repo/")

# Training: point-in-time-correct historical features
training_df = store.get_historical_features(
    entity_df=labels_df, # has user_id + event_timestamp + label
    features=["user_click_stats:click_count_7d", "user_click_stats:avg_session_length"],
).to_df()

# Serving: latest online features for one live request
online_features = store.get_online_features(
    features=["user_click_stats:click_count_7d"],
    entity_rows=[{"user_id": 42}],
).to_dict()

```

Part B — Great Expectations: build and run a validation suite

```

import great_expectations as gx
context = gx.get_context()

validator = context.sources.pandas_default.read_parquet(
    "data/credit_applications.parquet"
)

validator.expect_column_values_to_not_be_null("applicant_income")
validator.expect_column_mean_to_be_between("applicant_income", min_value=30000,
max_value=120000)
validator.expect_column_values_to_be_between("credit_score", min_value=300, max_value=850)
validator.save_expectation_suite("credit_applications_suite")

checkpoint = context.add_or_update_checkpoint(
    name="credit_pipeline_checkpoint",
    validator=validator,
)

```

```
)  
result = checkpoint.run()  
print("Validation success:", result.success)  
context.open_data_docs() # opens the HTML report
```

Lab target: every student registers at least one Feast feature view and successfully retrieves both historical (offline) and online features for the same entity, plus builds a Great Expectations suite with at least 4 expectations and runs it through a checkpoint, inspecting the generated Data Docs report.

References for This Module

- Chambers, Bill and Zaharia, Matei. Spark: The Definitive Guide. O'Reilly — primary reference for Sections 1.2–1.5.
- Pumperla, Max; Oakes, Edward; Liaw, Richard. Learning Ray. O'Reilly — primary reference for Section 2.
- Apache Spark official documentation: spark.apache.org/docs/latest/
- Ray official documentation: docs.ray.io
- Feast official documentation: docs.feast.dev
- Great Expectations official documentation: docs.greatexpectations.io